

The less commonly used `issubclass()` built-in function can identify whether the class object passed for the first argument is a subclass of (or the same class as) the class object passed for the second argument:

```
>>> issubclass(ChildClass, ParentClass) # ChildClass subclasses ParentClass.
True
>>> issubclass(ChildClass, str) # ChildClass doesn't subclass str.
False
>>> issubclass(ChildClass, ChildClass) # ChildClass is ChildClass.
True
```

As you can with `isinstance()`, you can pass a tuple of class objects as the second argument to `issubclass()` to see whether the first argument is a subclass of any of the classes in the tuple. The key difference between `isinstance()` and `issubclass()` is that `issubclass()` is passed two class objects, whereas `isinstance()` is passed an object and a class object.

Class Methods

Class methods are associated with a class rather than with individual objects, like regular methods are. You can recognize a class method in code when you see two markers: the `@classmethod` decorator before the method's `def` statement and the use of `cls` as the first parameter, as shown in the following example.

```
class ExampleClass:
    def exampleRegularMethod(self):
        print('This is a regular method.')

    @classmethod
    def exampleClassMethod(cls):
        print('This is a class method.')

# Call the class method without instantiating an object:
ExampleClass.exampleClassMethod()

obj = ExampleClass()
# Given the above line, these two lines are equivalent:
obj.exampleClassMethod()
obj.__class__.exampleClassMethod()
```
